AWS UG BENGALURU × HARNESS · AUTONOMOUS DEVOPS IN THE AGENTIC ERA

From failed deployment to fixed PR

Building an AI SRE agent on AWS + Kubernetes

Aditya Soni · Senior SRE, Warner Music Group

3 October 2026

BEFORE WE START

Hello, I'm Aditya

Senior SRE at Warner Music Group

AWS UG Leader @Jaipur, CNCF, OpenSearch and vCluster Ambassador

50+ talks worldwide, including KubeCon, Open Source Summit and many more

On call, like many of you



Three quick hands

1. Paged for a deploy that passed CI?

Three quick hands

1. Paged for a deploy that passed CI?
2. Pasted kubectl error output into chatgpt/cursor/claude?

Three quick hands

1. Paged for a deploy that passed CI?
2. Pasted kubectl error output into chatgpt/cursor/claude?
3. Would let an agent open a PR(Not Merge) on your repo?

01 · THE PROBLEM

11:42 PM. The rollout is stuck.

```
$ kubectl get pods -n payments
```

FAILED payment-api · rollout stuck · CreateContainerConfigError

FIXED fix/payment-api-secret-key · PR merged · 3/3 Ready

01 · THE PROBLEM

11:42 PM. The rollout is stuck.

```
$ kubectl get pods -n payments

NAME                              READY   STATUS                       RESTARTS
payment-api-5479c4d54d-frnjg      0/1     CreateContainerConfigError   0
payment-api-56586fb4b7-7lcnq      1/1     Running                      0
payment-api-56586fb4b7-k7rvn      1/1     Running                      0
payment-api-56586fb4b7-t2j5z      1/1     Running                      0
```

CI was green

Tests passed. The image was built and pushed.

Argo CD is not

Progressing, then Degraded. New pods never start.

The pager is yours

Old pods still serve traffic. For now.

01 · THE PROBLEM · YOUR TURN

You have the pager. Where do you look first?

A — **GitHub Actions**
Did the pipeline really pass?

B — **Argo CD**
What does the sync say?

C — **kubectl**
What are the pods doing?

D — **Grafana**
Are users affected?

E — **OpenSearch**
What do the logs say?

F — **git log**
What changed?

Shout your letter.

01 · THE PROBLEM

The honest answer: all six, in this order

1
GitHub Actions
Open the last run. Was it really green?

2
Argo CD
Which app, which sync, which revision?

3
kubectl describe
Pod status and events, one pod at a time

4
kubectl logs
Container output, if the container started

5
Prometheus
Write PromQL. Are users hurting yet?

6
OpenSearch
Search logs across pods and Regions

7
git log, git diff
What changed since it last worked?

8
The fix
Branch, edit, pull request, wait for CI

Eight steps across six tools, before the first line of the fix is written.

01 · THE PROBLEM

Four reasons this keeps hurting

Evidence is scattered

CI, the cluster, metrics, logs and Git each hold one piece. Nothing holds all five.

Context lives in heads

The runbook is whoever was on call the last time this happened.

The search is the work

The fix is often one line. Finding which line takes the incident.

Toil scales with deploys

More services and more Regions mean more of the same hunt, with the same people.

02 · THE IDEA

The fix is one line. Finding it is the job.

Give the reading to an agent. Keep the decision.

DEMO · PART 1 OF 3

Let's cause this incident now

1

Show it healthy

payment-api on Amazon EKS, 3/3 Ready

2

Merge the bad commit

It points at a Secret key that isn't there

3

Walk away

The agent is on call. We carry on talking.

We come back in ten minutes to see what it found.

02 · THE IDEA

From the page to the fix, it guides you

1 · DETECT

What broke, and where?

Service, Region and revision, the moment the alert fires

→

2 · TRIAGE

How bad is it?

User impact from live metrics, so you know how fast to move

→

3 · DIAGNOSE

Where do I look?

The tool that holds the answer, and the exact event, query or log line

→

4 · FIX

What do I do?

A ready pull request, a rollback, or the runbook step to take

→

5 · VERIFY

Did it work?

Health re-checked after the change, and the timeline written up

WHAT THE ON-CALL ENGINEER SEES, AS IT HAPPENS

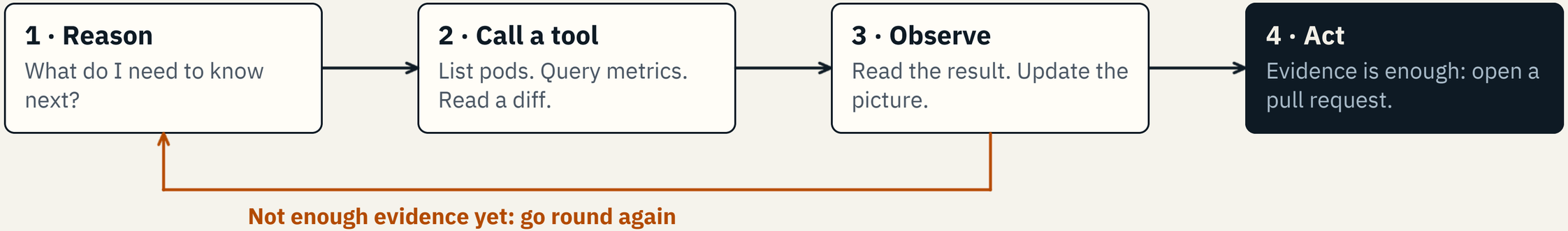
sre-agent · payment-api went Degraded after the last sync. Old pods still serving, no user impact yet.

sre-agent · Look at the pod events: couldn't find key DATABASE_URL in Secret payment-secret.

sre-agent · Cause: the last commit renamed the key. A fix is ready for your review.

02 · THE IDEA

An agent is a model in a loop with tools



A chatbot

You collect the logs and paste them in. It answers from what you chose to show it.

An agent

It collects its own evidence through tools, decides what to check next, and shows its work.

02 · THE IDEA

On AWS, this is a service: AWS DevOps Agent

THE MANAGED AGENT

AWS DevOps Agent

- Generally available since 31 March 2026
- Built on Amazon Bedrock AgentCore
- Investigates incidents across AWS, multicloud and on-prem
- Extends with MCP servers and custom skills

THIS SESSION

A look under the hood

- The same building blocks: Bedrock, AgentCore, MCP
- How an investigation works, step by step
- What to connect, and what to guard
- MCP servers and skills you can plug into DevOps Agent

Turn on the managed agent. Know it well enough to extend it.

Source: AWS What's New, "AWS DevOps Agent is now generally available", 31 March 2026

03 · THE TOOLS

The stack, layer by layer

Layer	Pick	Why
Model	Claude Sonnet 5 on Amazon Bedrock	Strong tool use; India geo profile stays in-country
Agent loop	Strands Agents, Python SDK	Open source, MCP client built in
Runtime	Bedrock AgentCore Runtime	Serverless agent sessions; a plain pod also works
Tools	Five MCP servers	No hand-written tool wrappers
Delivery	GitHub Actions → ECR → Argo CD	The pipeline the team already trusts
Signals	Prometheus + OpenSearch	Metrics and logs from both Regions
Platform	Amazon EKS in two Regions	Mumbai and Hyderabad

Sources: AWS ML Blog, Claude in India geographic cross-Region inference, Sept 2026 · strandsagents.com

03 · THE TOOLS

MCP: tools the agent discovers

WITHOUT MCP

Tools you write

```
def get_pods(): ...
def get_pod_logs(): ...
def query_prometheus(): ...
def create_pull_request(): ...
```

You write, test and maintain each one. Every API change is your problem.

WITH MCP

Tools the server ships

```
tools = server.list_tools_sync()
```

Names, schemas and descriptions arrive at runtime. The model reads them and chooses.

Maintained by the people who own each system: AWS, GitHub, Prometheus, OpenSearch.

Model Context Protocol: one open protocol between an agent and the systems it works on.

03 · THE TOOLS

Five MCP servers, no custom tool code

Server	Maintained by	The agent uses it to	Access
Amazon EKS MCP	AWS, managed, in preview	Read pods, events and logs; get troubleshooting guidance	Read-only
AWS MCP Server	AWS, managed, in the Agent Toolkit	Call any AWS API, search live docs, run sandboxed scripts	Scoped by IAM
GitHub MCP	GitHub	Read Actions runs, files and diffs; open a branch and a PR	PRs on one repo
Prometheus MCP	Prometheus project	Run PromQL, discover metrics	Read-only
OpenSearch MCP	OpenSearch project	Search log indices, read mappings	Read-only

Pin server versions for anything you demo or run in production. Tool names change between releases.

03 · THE TOOLS

Agent Toolkit for AWS: one command

```
$ aws configure agent-toolkit
```

AWS CLI 2.35+ · no additional charge · Kiro, Claude Code, Cursor, Codex and any MCP client

15,000+

AWS APIs, one MCP server

Plus live docs search and sandboxed scripts

40+

Agent skills

Tested procedures written by AWS

3

Plugins

aws-core, aws-agents, aws-data-analytics

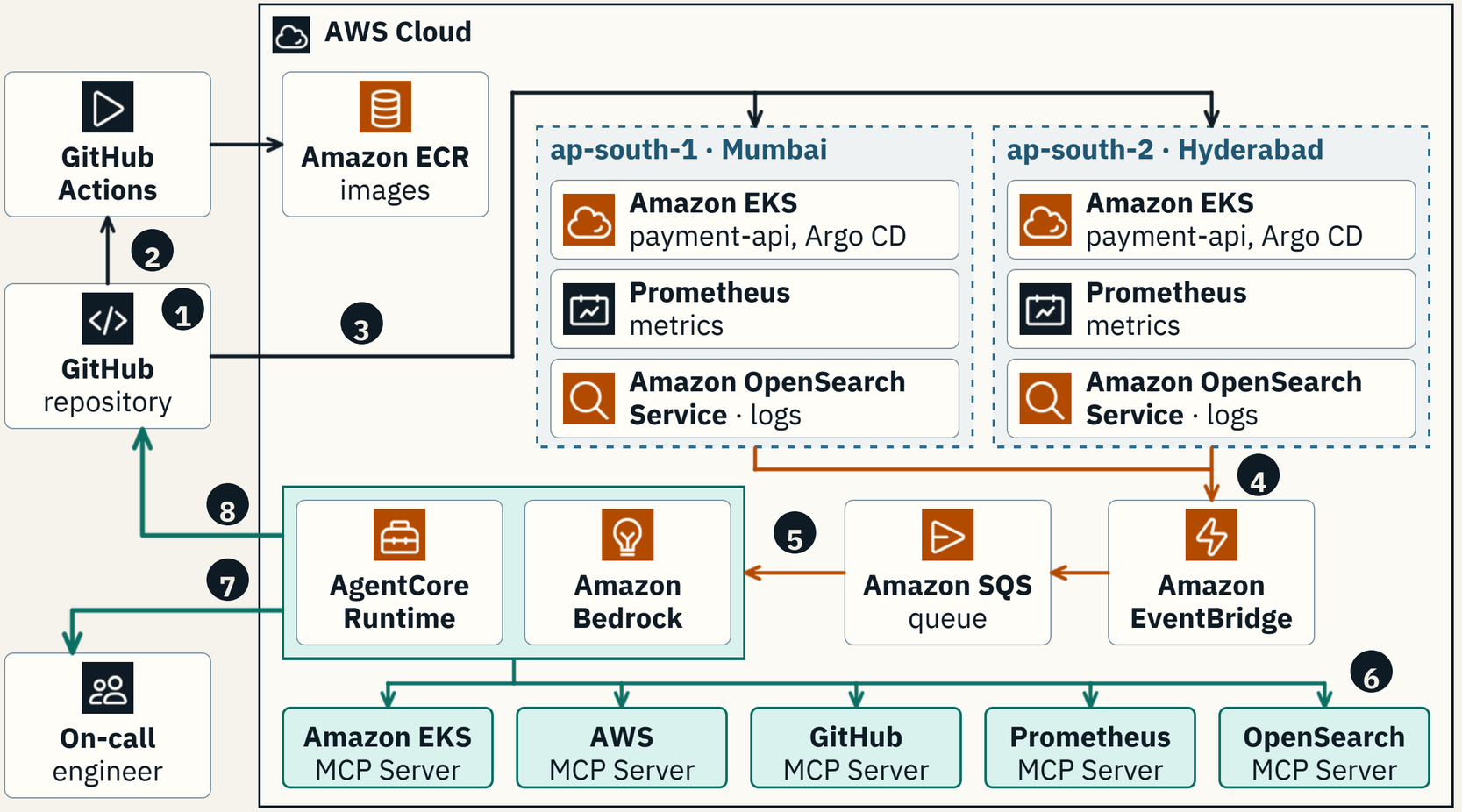
AT BUILD TIME

Your coding agent provisions EKS, ECR and IAM against current docs.

AT RUN TIME

The SRE agent calls AWS through the same server, under the same IAM.

Reference architecture: AI SRE agent on AWS



1. A change merges in GitHub
2. GitHub Actions pushes the image to Amazon ECR
3. Argo CD on each cluster syncs from Git
4. A failed rollout raises an event
5. The queue starts the agent run
6. Claude investigates through five MCP servers
7. The on-call engineer gets guidance as it goes
8. The fix arrives as a pull request

How the failure finds the agent

Argo CD Notifications
Degraded or sync failed

Alertmanager
error-rate alert firing

GitHub Actions
workflow run failed

→

1 · Intake
API Gateway and Lambda turn each webhook into one event shape

→

2 · EventBridge
A bus in each Region forwards to one central bus

→

3 · SQS queue
One investigation per incident, however many alerts fire

→

4 · Agent run
A Lambda invokes AgentCore Runtime with the event

```
{ "app": "payment-api", "region": "ap-south-1", "signal": "argocd.health.degraded", "revision": "a1b9f3c" }
```

The event says where and what. The agent's first prompt is built from it.

DEMO · PART 2 OF 3 · YOUR TURN

Before we look: what's your diagnosis for CreateContainerConfigError?

A The image can't be pulled

B A Secret or ConfigMap key is missing

C The container ran out of memory

D The readiness probe is failing

Hands up for A, B, C, D. Then we ask the agent.

04 · STAGES 2 AND 3 OF 5 · TRIAGE, DIAGNOSE

Seven lookups, one conclusion

1	GitHub MCP	Latest commits to the manifests	"align secret key name with env var", minutes ago
2	EKS MCP	Pods in namespace payments	One new pod 0/1, CreateContainerConfigError
3	Prometheus MCP	Available replicas	3 old replicas still serving. No user impact yet.
4	EKS MCP	Events in the namespace	couldn't find key DATABASE_URL in Secret payments/payment-secret
5	OpenSearch MCP	Log lines from the new pod	Zero: the container never started
6	GitHub MCP	Diff of that commit	key: DB_URL became key: DATABASE_URL
7	GitHub MCP	secret.yaml in the repo	The Secret defines only DB_URL

ROOT CAUSE

The Deployment references a Secret key that does not exist. The last commit introduced it.

A pull request that carries its evidence

k8s/payment-api/deployment.yaml

```
 - name: DATABASE_URL
   valueFrom:
     secretKeyRef:
       name: payment-secret
-      key: DATABASE_URL
+      key: DB_URL
```

One line. The smallest change that explains every piece of evidence.

Pull request · opened by sre-agent

fix(payment-api): restore Secret key DB_URL for DATABASE_URL

Root cause. The Deployment references key DATABASE_URL, which does not exist in payment-secret.

Evidence.

- Pod event: couldn't find key DATABASE_URL
- Commit diff: key DB_URL was renamed
- secret.yaml in the repo defines only DB_URL

Not done. No cluster changes. No Secret values read.

Needs 1 human review

DEMO · PART 3 OF 3 · YOUR CALL

You are the reviewer. Do we merge it?

1

Read the evidence

Root cause, three facts, a one-line diff

2

Votes

Approve or decline.

3

Merge and watch

Argo CD syncs, pods go Ready, the agent confirms

The agent proposed. The decision is ours.

04 · STAGE 5 OF 5 · VERIFY

The pipeline decides. The agent checks.

1

CI runs on the PR

Manifest validation and tests, same as any change

2

A human merges

Branch protection requires one approval

3

Argo CD syncs

One Region first, then the other

IF HEALTHY

Close the loop

The agent re-checks pods and error rate in both Regions, then posts the timeline on the PR.

IF STILL UNHEALTHY

Hand over

It posts the new evidence, suggests a git revert, and pages a human. One attempt, no retry loop.

04 · THE ARCHITECTURE

Two Regions: what runs where

Piece	Where	Worth knowing
App, Prometheus, OpenSearch	ap-south-1 and ap-south-2	Same manifests; Argo CD syncs both
Model inference	India geo inference profile	Routes only between Mumbai and Hyderabad
Agent runtime	One Region	Events from both Regions land on one bus
EKS MCP Server	Regional endpoint per cluster	In preview: check it exists in your Region
AWS MCP Server	us-east-1 endpoint	Calls AWS APIs in any Region

THE AGENT'S FIRST QUESTION

One Region failing points at infrastructure. Both failing after a commit points at configuration.

Sources: AWS ML Blog on India cross-Region inference · EKS User Guide · Agent Toolkit for AWS setup guide

04 · THE ARCHITECTURE

The whole agent fits on a slide

```
from strands import Agent
from strands.tools.mcp import MCPClient
eks = MCPClient(aws_mcp("eks-mcp", REGION, read_only=True))
aws = MCPClient(aws_mcp("aws-mcp", "us-east-1"))
github = MCPClient(http_mcp(GITHUB_MCP_URL, token=GH_TOKEN))
prom = MCPClient(stdio_mcp("prometheus-mcp-server"))
logs = MCPClient(stdio_mcp("opensearch-mcp-server-py"))
with eks, aws, github, prom, logs:
    servers = (eks, aws, github, prom, logs)
    tools = [t for s in servers for t in s.list_tools_sync()]
    agent = Agent(model=MODEL, tools=tools, system_prompt=SRE_RULES)
    agent(f"{app} is Degraded in {region}. Find the cause."
          " Open a fix PR. Change nothing else.")
```

aws_mcp, http_mcp and stdio_mcp are three-line helpers around the SigV4 proxy, streamable HTTP and stdio transports.

05 · GUARDRAILS

What the agent is allowed to touch

IT CAN READ

- Pods, events, logs
- Metrics and log indices
- The repo and CI runs
- AWS resource state

Enforced by the read-only flag on the EKS MCP proxy and a read-only IAM role

IT CAN PROPOSE

- A branch
- A commit
- A pull request
- A comment on that PR

Enforced by a GitHub token scoped to one repo, and branch protection

IT CANNOT

- Apply anything to a cluster
- Read Secret values
- Merge its own PR
- Delete or scale resources

Enforced by having no credentials for it, plus an IAM deny on agent calls

Plus a turn limit, a token budget, and CloudTrail for every AWS call the agent makes.

05 · GUARDRAILS

IAM can tell the agent from you

```
{
  "Effect": "Deny",
  "Action": [
    "eks:Delete*", "ec2:Terminate*",
    "iam:*", "secretsmanager:GetSecretValue"
  ],
  "Resource": "*",
  "Condition": { "Bool": {
    "aws:ViaAWSMCPService": "true"
  } }
}
```

`aws:ViaAWSMCPService`

True when a request arrives through an AWS-managed MCP server

`aws:CalledViaAWSMCP`

Names the server: aws-mcp.amazonaws.com or eks-mcp.amazonaws.com

Same role, stricter rules when an agent is driving.

Source: AWS Security Blog, "Understanding IAM for Managed AWS MCP Servers"

07 · TAKE HOME

Three things to try on Monday

1

Install the Agent Toolkit in a sandbox account

`aws configure agent-toolkit`

2

Point your coding agent at one cluster, read-only

Add the EKS MCP Server and ask it why a pod is failing

3

Pick one failure class and write the PR-only agent for it

Bad config references are a good first one: common, small, easy to verify

THANK YOU · QUESTIONS

Get paged.
Get guided.
Get back to sleep.



linkedin.com/in/adityasonittyl/